ERP Data Migration Tool

User Guide

Copy data from the old ERP export into the new ERP's column layout — safely, and without typing anything by hand.

A step-by-step guide — no programming experience needed

1 What This Tool Does

This tool copies data from the **old ERP's** Excel export into a brand-new Excel file laid out the way the **new ERP** expects it. You tell it, once, which old column goes into which new column — the tool does the copying.

How it works, in plain terms

1. You open a small Excel file called **data_migration_helper.xlsx** and write down which old column name matches which new column name.
2. You run the tool and point it at your old ERP export file.
3. The tool reads the mapping, creates a brand-new Excel file, and copies every value across exactly as it is — dates stay dates, codes keep their leading zeros, and numbers are never rounded.
4. Before it saves anything, it double-checks every single copied cell against the old file. If even one value doesn't match, it stops and creates nothing.

> **The old file is never touched**
>
> The tool only *reads* the old ERP file and the mapping file. It never changes them, and it never overwrites an existing new file unless you explicitly say so.

What's in this folder

- `main.py` — the tool itself. Don't open or change it.
- `data_migration_helper.xlsx` — the mapping list. You fill this in.
- `requirements.txt` — a list of helper packages the tool needs. Used automatically during setup.
- `test_data` — practice files with made-up data, so you can try the tool safely before using it on real data.
- `README.md` and `ERP_Data_Migration_User_Guide.pdf` — the instructions (this guide). Every update brings the newest version.
- `.git` (a hidden folder) and `.gitignore` — used by Git to keep track of updates. Don't delete or change them.

2 Installation (One Time, On Each Computer)

Do this once on every computer that will run the tool. It takes about 10–15 minutes. You'll install two free programs — **Python** (it runs the tool) and **Git** (it downloads the tool and brings you updates) — then download the tool and set it up. You'll use the black Command Prompt window — don't worry, every step tells you exactly what to type.

Step 2.1 — Check if Python is already installed

Click **Start**, type `cmd` and press Enter. A black window opens — this is the Command Prompt. Type:

```
python --version
```

- If it shows something like `Python 3.13.5` (3.8 or higher), Python is already installed — skip to Step 2.3.
- If it says *'python' is not recognized*, or the Microsoft Store opens, or the version is below 3.8, install Python using Step 2.2.

Step 2.2 — Install Python (skip if Step 2.1 found it)

1. Go to `https://www.python.org/downloads/windows/`
2. Under the newest **Stable Release**, click **Windows installer (64-bit)**. Do not choose the "embeddable package" — it will not work for this tool.
3. Open the file you downloaded.
4. On the very first screen, **tick the box "Add python.exe to PATH"** at the bottom — this step matters. Then click **Install Now**.
5. When it finishes, click **Close**. If it offers **Disable path length limit**, click that too.
6. Close the Command Prompt window completely and open a new one. Type `python --version` again — it should now show a version number.

> **If the install asks for an administrator password**
>
> Ask your IT team to approve or run the installer. Don't skip the "Add to PATH" tick box — without it, none of the later steps will work.

> **If python --version still doesn't work, but py --version does**
>
> That's fine — Python installed without the PATH option. Just use `py` instead of `python` in Step 2.6 below (for example, `py -m venv .venv`). Everything else works exactly the same.

Step 2.3 — Install Git

Git is a free program that downloads the tool from **GitHub** (the website where the tool is kept) and later brings you updates with one command. You'll install it with `winget`, the app installer that comes with Windows.

1. Open the Command Prompt (**Start**, type **cmd**, press Enter) and check that winget is available:

```
winget --version
```

- If it shows a version number like `v1.9.25200`, winget is ready — go on to the next step.
- If it says *'winget' is not recognized*, see the box below these steps.

2. Install Git by typing this line and pressing Enter:

```
winget install --id Git.Git -e --source winget
```

In plain words, this means: *install exactly the official Git package from Microsoft's app list.*

- If it asks *Do you agree to all the source agreements terms?*, type `Y` and press Enter.
- If Windows asks *Do you want to allow this app to make changes to your device?*, click **Yes**. If it asks for an administrator password, ask your IT team to approve it.
- Wait until it says `Successfully installed`. This can take a minute or two.

3. **Close the Command Prompt window completely and open a new one.** A window that was already open doesn't know Git was just installed.

4. Check that Git works:

```
git --version
```

It should show something like `git version 2.47.1.windows.1`. Any version number is fine.

> **If 'winget' is not recognized**
>
> winget comes with Microsoft's **App Installer**. Open the **Microsoft Store**, search for **App Installer**, click **Update** (or **Get**), then open a new Command Prompt and try again. If the Store is blocked on your company computer, ask IT to install Git for you — or download it from `https://git-scm.com/downloads/win` and click **Next** on every screen to keep the default options.

> **If git --version says 'git' is not recognized**
>
> Close every Command Prompt window and open a new one. Still failing? Restart the computer and try again. If it still doesn't work, the install didn't finish — type the `winget install` line again.

Step 2.4 — Download the tool ("clone")

Cloning means downloading your own copy of the tool from GitHub. You do this only once on each computer — after that, you get updates with `git pull` (Section 4).

1. Open a new Command Prompt (**Start**, type **cmd**, press Enter). It opens in your personal folder, for example `C:\Users\YourName`.

2. Type this line and press Enter:

```
git clone https://github.com/akshaysanil/erp_data_migration.git
```

3. Go into the new folder:

```
cd erp_data_migration
```

What happens when you clone

- Git creates a new folder called **erp_data_migration** inside your personal folder and downloads all the tool's files into it.

- You'll see lines like `Cloning into` ... and `Receiving objects: 100%`. When the blinking cursor comes back, it's finished.
- The new folder stays linked to GitHub — that's what lets you get updates later with one command.
- You don't need a GitHub account or password to download it.

Your tool's folder is now `C:\Users\YourName\erp_data_migration`. To open it in File Explorer, type `%USERPROFILE%\erp_data_migration` in the address bar and press Enter.

> **If it says destination path 'erp_data_migration' already exists**
>
> The tool is already downloaded on this computer. Don't clone it again — go to Step 2.5. To get the newest version, use `git pull` (Section 4).

> **Used the zip version before?**
>
> A folder unzipped from the old zip file can't receive updates. Clone a fresh copy as shown above. If you had already filled in your mapping, copy `data_migration_helper.xlsx` from the old folder into the new one (replace the file). Once the tool works from the new folder, you can delete the old one.

Step 2.5 — Open a Command Prompt inside the tool's folder

If you just finished Step 2.4, you're already there — go on to Step 2.6. Any other time, use one of these:

- **From File Explorer:** open the tool's folder, click once on the address bar at the top, type `cmd`, and press Enter. A black window opens, already inside that folder.
- **From the Start menu:** open the Command Prompt and type `cd erp_data_migration`

Step 2.6 — Create the environment and install the packages

Type each of these three lines, pressing Enter after each one:

```
python -m venv .venv
.venv\Scripts\activate
python -m pip install -r requirements.txt
```

- After the second line, the prompt starts with `(.venv)` — that means the environment is switched on.
- The third line downloads the packages, so it needs an internet connection. It finishes with a line like `Successfully installed ... openpyxl ...`

> **Do this on every computer**
>
> A `.venv` folder copied from another computer will not work. All of Section 2 must be done on each machine that will run the tool.

If something goes wrong during installation

- **"No module named pip"** — type `python -m ensurepip --upgrade`, then run the install line again.
- **"'pip' is not recognized"** — type `python -m pip` instead of just `pip`. Make sure the line starts with `(.venv)` first; if not, run `.venv\Scripts\activate`.

- **Connection, proxy or SSL error** — the computer can't reach the internet to download packages. Check the connection, or ask IT if you're on a company network.
- **"A new release of pip is available"** — this is just a notice. You can ignore it.

3 Instructions — Running a Migration

Step 3.1 — Fill in the mapping

Open `data_migration_helper.xlsx` and go to the **Mapping** sheet. Each row becomes one column of the new file, in the same order you list them:

- **New Column** — the column name the new system needs. Each name may only appear once.
- **Old Column** — the matching column name exactly as it appears in the old file. Capitalisation and extra spaces don't matter. Leave it blank to create an empty column in the new file. The same old column can feed more than one new column.
- **Notes** — for yourself only. The tool ignores this column.

> **Before your first real run**
>
> Delete the **EXAMPLE** rows. Don't rename the Mapping sheet or the column headers in row 1. Save and close the file before continuing.

Step 3.2 — Run the tool

1. Close the old Excel file if it's open.
2. Open a Command Prompt in the tool's folder (Step 2.5) and switch the environment on — needed every time you open a new window:

```
.venv\Scripts\activate
```

Then run the tool in one of two ways:

Way A — the tool asks you questions (easiest)

```
python main.py
```

It will ask for the old file, the sheet (only if there's more than one), the row with the column names, the mapping file, and the name of the new file. To give a file, simply drag it from File Explorer into the black window and press Enter. To accept a suggestion shown in [brackets], just press Enter. At the end it shows your answers and waits for Enter before starting.

Way B — give all the details in one line

```
python main.py --old "C:\Data\old_data.xlsx" --output "C:\Data\new_data.xlsx"
```

- `--old` — the old file with the data.
- `--output` — the new file to create.

- Put **"quotes"** around every file path.
- To get a file's exact path: right-click it in File Explorer and choose **Copy as path** (hold Shift while right-clicking on older Windows), then paste it into the black window.

Add these extra options only when you need them:

- `--sheet "Customers"` — the old file has more than one sheet; give the name of the tab that has the data.
- `--header-row 3` — the column names aren't in row 1; give the correct row number.
- `--helper "C:\Data\other_mapping.xlsx"` — use a different mapping file instead of the one in the tool's folder.
- `--new-sheet "Import"` — name the sheet in the new file (default is Sheet1).
- `--overwrite` — replace the new file if it already exists. Without it, the tool stops instead of overwriting.

```
python main.py --old "C:\Data\old_data.xlsx" --sheet "Customers" --header-row 3
    --output "C:\Data\new_data.xlsx"
```

Example: data on the “Customers” tab, column names on row 3.

> **Want the full list of options explained?**
>
> Type `python main.py --help`

> **Never create the new file yourself**
>
> The tool creates the new Excel file. If a file with that name already exists, the tool stops rather than guessing what you want — add `--overwrite` if you really want to replace it.

Step 3.3 — Check the result

When it finishes, the window shows a summary like this:

```
Old file         : C:\Data\old_data.xlsx (header row 1)
Columns          : 13 in new file, 12 filled from old file, 1 left empty
Rows copied      : 15
Verified         : 195 cells match the old file exactly
Saved to         : C:\Data\new_data.xlsx

New columns left empty (no Old Column in the helper): branch

Old columns NOT migrated (not in the helper): Sales Rep, Internal Notes
```

> **Always read the “NOT migrated” list**
>
> Those old columns did not make it into the new file because they weren't listed in the mapping. Double-check that is really what you want before relying on the result. Finally, open the new file and glance over a few rows yourself.

Practice first with the test files

Before touching real data, try the tool on the made-up practice files in the test_data folder:

```
python main.py --old test_data\old_erp_data.xlsx --helper
    test_data\test_mapping_helper.xlsx --output test_data\practice_result.xlsx
```

Compare the result, `test_data\practice_result.xlsx`, against `test_data\expected_new_erp_data.xlsx` — they should match. The summary should report 15 rows, *branch* left empty, and *Sales Rep, Internal Notes* not migrated. To run the practice again, delete the output file first, or add `--overwrite`.

4 Getting Updates

The official copy of the tool is kept on GitHub. When you need a change, you ask for it; once it's ready, one command brings it to your computer. Your own files and your setup stay as they are.

How updates work

1. **Ask for the change.** Tell the tool's maintainer what you need — a new feature, something that should work differently, or an error you saw. Say what you were doing and copy the exact message from the black window.
2. **Wait for the go-ahead.** The maintainer makes the change, uploads it to GitHub, and lets you know it's ready.
3. **Get the update.** Run `git pull` (Step 4.1) on each computer that uses the tool.

> **Never send real ERP data**
>
> When asking for a change, don't attach real ERP files, and don't send screenshots that show real data. The tool's messages only show column names and cell positions, so copying a message's text is safe. If an example file is needed, use the practice files in test_data or a small file with made-up data.

Step 4.1 — Download the latest version

1. Close the tool's files if they're open in Excel — especially `data_migration_helper.xlsx`.
2. Open a Command Prompt in the tool's folder (Step 2.5) and type:

```
git pull
```

What you'll see

- `Already up to date.` — you already have the newest version. Nothing was changed.
- A list of file names ending with a line like `1 file changed, 12 insertions(+), 6 deletions(-)` — the update was downloaded, and the files listed were replaced with their newest versions.

```
From https://github.com/akshaysanil/erp_data_migration
   bf70b4d..3c9e1a2  main       -> origin/main
Updating bf70b4d..3c9e1a2
Fast-forward
 main.py | 18 ++++++++++++------
 1 file changed, 12 insertions(+), 6 deletions(-)
```

Example: one file, main.py, was updated. The codes and numbers will be different each time.

> **What git pull changes — and what it leaves alone**
>
> It only replaces the tool's own files that were updated, such as main.py or this guide. It never touches your `.venv` folder or files you created yourself. Your filled-in `data_migration_helper.xlsx` is kept too — unless the update changes that same file; then Git stops and tells you (see Step 4.3). It only *downloads* — nothing on your computer is ever uploaded.

Step 4.2 — Update the packages

After every pull, switch the environment on and run the install line again:

```
.venv\Scripts\activate
python -m pip install -r requirements.txt
```

If nothing new is needed, it only prints lines starting with `Requirement already satisfied` — that's fine. Then use the tool as usual (Section 3).

Step 4.3 — If git pull stops with "would be overwritten"

You may see a message like this:

```
error: Your local changes to the following files would be overwritten by merge:
        data_migration_helper.xlsx
Please commit your changes or stash them before you merge.
Aborting
```

This means the update changes a file that you have also changed on this computer — usually your filled-in mapping. Git stopped so it wouldn't erase your work. **Nothing was changed.** To fix it:

1. In File Explorer, **move** each file named in the message out of the tool's folder — for example, drag it to your Desktop. Move it; don't copy it.
2. Run `git pull` again. This time it finishes, and the newest version of the file comes down.
3. If the file you moved was your mapping, open it next to the new `data_migration_helper.xlsx` and copy your rows into the new file, then save. Copy the rows rather than replacing the whole file — the update may have changed it.

Other messages you might see

- **'git' is not recognized** — Git isn't installed on this computer, or this window was opened before Git was installed. Open a new Command Prompt; if it still fails, do Step 2.3.
- **fatal: not a git repository** — you're not inside the tool's folder, or the folder came from the old zip file. Open the Command Prompt in the tool's folder (Step 2.5). If the folder came from the zip, follow

Step 2.4 to download a fresh copy.

- **Could not resolve host: github.com** or **unable to access** — the computer can't reach GitHub. Check the internet connection, or ask IT if you're on a company network.
- **Unlink of file ... failed. Should I try again? (y/n)** — that file is open in Excel. Close it, then type `y` and press Enter.
- **Anything else** — don't try other Git commands. Copy the message and send it to the tool's maintainer.

> **Two habits that keep updates trouble-free**
>
> Use only `git clone` (once per computer) and `git pull` (for updates) — don't run other Git commands unless the maintainer asks you to. And keep real ERP files, and the new files you create, outside the tool's folder — for example in `C:\Data`.

5 If You See an Error

When the tool stops with an error, it hasn't created anything yet. Fix the problem below and run it again. (For messages from Git, see Step 4.3.)

- **'python' is not recognized, or the Microsoft Store opens** — the environment isn't switched on. Run `.venv\Scripts\activate` first. Still failing? Make sure you're in the tool's folder and that Step 2.6 was completed on this computer.
- **'No module named openpyxl'** — same fix: run `.venv\Scripts\activate` first.
- **'running scripts is disabled on this system'** — you're in PowerShell, not Command Prompt. Type `cmd`, press Enter, and try again.
- **'…already exists; use --overwrite'** — a file with that name already exists. Choose another name, or add `--overwrite`.
- **'old column is not in row 1 … did you mean …'** — the Old Column name in the mapping is spelled differently from the old file. Fix the spelling, or add `--header-row` if the column names aren't in row 1.
- **'old column … appears more than once'** — two columns in the old file share a name. Rename one of them in a copy of the old file.
- **'new column … is already used in row N'** — the same New Column name was typed twice in the mapping.
- **'…has N sheets … choose one with --sheet'** — add `--sheet "SheetName"`.
- **'cannot open … as an .xlsx file'** — the file is an old-style .xls file, or is damaged. Open it in Excel and use **Save As** → **Excel Workbook (.xlsx)**.
- **'formulas whose result was never saved'** — open the old file in Excel, save it, close it, and run the tool again.
- **'Permission denied … close the file if it is open in Excel'** — the new file is currently open in Excel. Close it and run again.

6 Safety — What the Tool Will Never Do

- It never changes the old file or the mapping file — both are only read.
- It checks every copied cell against the old file before saving. If anything doesn't match exactly, no new file is created.
- It never replaces an existing file unless you add `--overwrite`, or answer yes when asked.
- Its messages only ever show column names and cell positions (like `C14`) — never the actual data.
- Blank rows in the middle of the data are preserved in their original position; only blank rows at the very end are dropped.

Questions about the mapping or a result you're not sure about? Ask before running the tool on real ERP data — it's always safer to double-check first.